

=====

==

CPSC 3750 — Exam 1: AI Reflection & Feature Documentation

=====

==

=====

==

PART 1 — AI REFLECTION: Hunt/Target AI with Memory (10 Points)

=====

==

FEATURE DESCRIPTION

The Hunt/Target AI is an intelligent computer opponent that uses a two-phase strategy to find and destroy the player's ships. In "hunt mode," the AI fires at random unfired cells to discover ships. When it scores a hit, it switches to "target mode," queuing the four adjacent cells (up, down, left, right) as priority targets. The AI continues targeting until it misses or sinks a ship. When a ship is fully sunk, the target queue is cleared so the AI returns to hunting for the next undiscovered ship.

WHY I CHOSE THIS FEATURE

This is my favorite feature because it transforms the game from a trivial exercise (random shooting has no strategy) into a genuinely challenging opponent. The AI feels intelligent — it "remembers" where it hit and

systematically destroys ships instead of wandering aimlessly. It also demonstrates real state reasoning: the AI must track which cells have been fired, which are hits, maintain a target queue, and detect when a ship is sunk to reset its strategy. This is the feature where architecture decisions mattered most.

WHERE STATE LIVES

- Server (authoritative): The ``game`` object in `server.js` contains ``aiTargetQueue`` (array of "r,c" strings), ``yourHits``, and ``yourMisses``. These three pieces of state together drive the AI's decision-making.
- Persistence: The entire game object, including ``aiTargetQueue``, is saved to ``game-state.json`` after every state change. On server restart, the AI resumes exactly where it left off — mid-hunt or mid-target.
- Client: The client never sees ``aiTargetQueue`` or ``enemyShips``. It only receives ``yourHits`` and ``yourMisses`` to render the player's grid. The AI logic runs entirely server-side.

HOW I USED AI TO BUILD IT

I used AI (Cursor/Claude) as an implementation accelerator. I described the desired behavior at a high level and let AI generate the initial code, then reviewed and modified the output to ensure correctness.

HOW MY PROMPTS EVOLVED

My prompts started broad and became more specific as I discovered edge cases and integration issues. The key evolution was from requesting a feature to debugging its behavior within the existing state machine.

ANNOTATED PROMPT LOG

Prompt 1:

"Add the Advanced AI with hunt/target behavior and memory of hits."

Why I asked this:

The basic AI was pure random — not fun to play against. I wanted a smarter opponent that felt like it was "thinking."

What AI produced:

A `runComputerTurn()` function with a target queue, `addNeighborsToTargetQueue()` helper, and modified game state to include `aiTargetQueue`. The AI checks the queue first (target mode), falls back to random (hunt mode).

Issue discovered:

The AI would keep targeting cells from a previous ship after sinking it, wasting shots on empty water near an already-destroyed ship.

Prompt 2:

"When a ship is sunk, clear the target queue so the AI goes back to hunt mode."

Why I asked this:

Without clearing the queue on sunk ships, the AI behaved inefficiently — it would fire at neighbors of a fully destroyed ship instead of hunting for new ones.

What AI produced:

An `anyShipSunk()` function that checks if all cells of any player ship are in the hits set, and a queue-clear call after each hit: `if (anyShipSunk()) game.aiTargetQueue = [];`

Issue discovered:

The `anyShipSunk()` function returns true if ANY ship is fully hit, not just the one that was JUST sunk. This means after the second ship is sunk, it still detects the first (already sunk) ship and clears the queue prematurely. In practice this works acceptably because the queue is only relevant when actively targeting, but it's technically imprecise.

Prompt 3:

"Make sure the AI target queue persists across server restarts and is cleared on new game / restart battle."

Why I asked this:

If the server restarts mid-game, the AI would lose its targeting memory and revert to random shooting even if it was in the middle of destroying a ship.

What AI produced:

Added `aiTargetQueue`` to the ``game`` object (already persisted via ``saveState()``), added a guard in ``loadState()`` to default it to an empty array if missing, and added explicit clears in the new game and restart endpoints.

MANUAL MODIFICATIONS

1. Verified that `aiTargetQueue`` was properly initialized as an empty array in the default game state (AI had it but I double-checked the type guard in `loadState``).
2. Reviewed the `addNeighborsToTargetQueue`` function to confirm it checks both the "already fired" set AND the "already in queue" set to prevent duplicates.
3. Traced the state machine flow to ensure `COMPUTER_TURN` loops correctly: the server runs `runComputerTurn()`` in a while loop until the state is no longer `COMPUTER_TURN` (i.e., the AI missed or the game ended). This was already in the fire endpoint but I verified the loop termination conditions.
4. Tested edge case: what happens when the AI's target queue contains cells that were fired at between turns (e.g., after a restart). Confirmed the `firedSet.has(k)`` check in the queue-draining loop handles this correctly.

=====

==

PART 2 — NEW FEATURE 1: Persistent Scoreboard (JSON-Based) — 20 Points

=====

==

FEATURE DESCRIPTION

A persistent scoreboard that tracks the player's combat record across all games.

Statistics tracked include: total wins, total losses, total games played, shot accuracy (hits/total shots), best game (fewest shots to win), current win streak, longest win streak, and a history of the last 20 games with per-game details (result, shots, accuracy, difficulty, date).

The scoreboard is displayed in a "COMBAT RECORD" panel below the game grids, styled consistently with the radar theme. It includes six stat cards and a scrollable game history list. A "RESET" button allows clearing all stats.

ARCHITECTURAL EXPLANATION (Where State Lives)

- Server (authoritative): The ``scoreboard`` object in `server.js` holds all stats. It is completely separate from the game state — stored in ``scoreboard.json``, not ``game-state.json``.
- Persistence: ``scoreboard.json`` is read on server startup (``loadScoreboard()``) and written after every game completion (``saveScoreboard()``). File operations use ``fs.openSync``/``fs.writeFileSync``/``fs.closeSync`` for safe writes.
- Client: Fetches scoreboard via ``GET /api/scoreboard`` on page load and after each game ends. The client only displays data — it never writes to the scoreboard directly.
- Data flow: When a game reaches `GAME_OVER`, the fire endpoint calls ``recordGameResult(winner)`` which updates the in-memory scoreboard, then calls

``saveScoreboard()`` to persist. The client's ``syncUI()`` detects `GAME_OVER` and triggers ``fetchScoreboard()`` to refresh the display.

WHY THIS USES PERSISTENT STORAGE CORRECTLY

- Data persists across browser refresh (scoreboard.json is on the server)
- Data persists after server/Apache restart (loaded from file on startup)
- Server-side read/write logic handles all mutations
- No manual file editing required — scoreboard.json is auto-created with defaults if it doesn't exist
- File locking via `fd open/write/close` pattern prevents corruption

ANNOTATED PROMPT LOG

Prompt 1:

"Help me implement a JSON-based scoreboard for my Node.js Battleship game that tracks wins, losses, accuracy, and best game. It should persist in a separate JSON file from the game state."

Why I asked this:

I needed a persistent feature that goes beyond the existing game-state.json.

A scoreboard is a natural fit — it accumulates data across multiple games and clearly demonstrates persistence.

What AI produced:

Initial implementation with ``scoreboard.json``, ``loadScoreboard()``, ``saveScoreboard()``, ``recordGameResult()``, and a GET endpoint. Used basic ``fs.writeFileSync`` for persistence.

Issue discovered:

The initial ``recordGameResult`` was called inside both the player-win and AI-win code paths, but the AI-win path recorded the result BEFORE the while-loop for computer turns had fully resolved. This meant the shot counts were wrong when the AI won.

Prompt 2:

"Move the `recordGameResult` call for AI wins to AFTER the computer turn loop completes, so the final shot counts are accurate."

Why I asked this:

The AI fires multiple shots per turn (while loop continues until miss or game over). Recording stats mid-loop captured incomplete data.

What AI produced:

Moved the ``recordGameResult('computer')`` call to after the while loop, with a check: ``if (game.state === STATE.GAME_OVER && game.winner === 'computer')``.

Prompt 3:

"Add game history tracking — store the last 20 games with result, shots, hits, difficulty, and date. Display them in a scrollable list in the UI."

Why I asked this:

Raw totals (wins/losses) aren't very interesting. Per-game history lets the player see trends and compare performance across difficulty levels.

What AI produced:

Added `gameHistory` array to scoreboard with push/slice logic to keep last 20 entries. Added client-side rendering in `syncScoreboard()`.

Prompt 4:

"Add file locking to the scoreboard save to prevent corruption if the server crashes mid-write."

Why I asked this:

Basic `writeFileSync` can leave a corrupted file if the process is killed mid-write. Using `openSync`/`writeFileSync(fd)`/`closeSync` is safer.

What AI produced:

Updated `saveScoreboard()` to use file descriptor pattern.

MANUAL CHANGES

1. Added a merge with `DEFAULT_SCOREBOARD` in `loadScoreboard()` so that if new fields are added in future versions, they get default values instead of being undefined.
2. Added the accuracy calculation to the `GET /api/scoreboard` endpoint rather than storing it — accuracy is derived from `totalPlayerHits/totalPlayerShots`,

so storing it would create a data consistency risk.

3. Verified that the reset endpoint creates a fresh copy of DEFAULT_SCOREBOARD with a new empty array for gameHistory (not a reference to the default).
4. Added guard against double-recording: ensured recordGameResult is only called once per game end by structuring the code paths so player-win returns early and AI-win is checked after the loop.

=====

==

PART 2 — NEW FEATURE 2: AI Difficulty Levels — 15 Points

=====

==

FEATURE DESCRIPTION

Three selectable AI difficulty levels that fundamentally change how the computer opponent plays:

- EASY: Pure random targeting. The AI picks any unfired cell at random with no memory or intelligence. It does NOT queue adjacent cells after a hit. This makes the AI significantly weaker — it may hit a ship and then wander away.
- MEDIUM: Hunt/target with memory (the original AI). After a hit, the AI queues adjacent cells and prioritizes them. When a ship is sunk, the queue is cleared. This is the default difficulty.

- HARD: Enhanced hunt/target with probability-aware targeting and checkerboard hunting. When hunting (no targets), the AI fires on checkerboard cells (where $(\text{row} + \text{col}) \% 2 === 0$) to maximize coverage with fewer shots. When targeting, it scores each queued cell by how many adjacent hits exist and whether they form a line, then picks the highest-scored cell. This makes the AI significantly more efficient at finding and destroying ships.

Difficulty can be changed during SETUP (ship placement phase) via buttons in the header. The selected difficulty persists in the game state and is displayed in the status bar during gameplay.

ARCHITECTURAL EXPLANATION (Where State Lives)

-
- Server (authoritative): `game.difficulty` is a string field ('easy', 'medium', 'hard') stored in the game object. The `runComputerTurn()` function dispatches to `runComputerTurnEasy()`, `runComputerTurnMedium()`, or `runComputerTurnHard()` based on this value.
 - Persistence: Difficulty is part of the game object, so it persists in `game-state.json` along with all other game state. If the server restarts mid-game, the AI continues playing at the same difficulty.
 - Client: Receives `difficulty` in every `clientState()` response. The UI syncs the active button state and shows the difficulty label in the status bar. Difficulty changes are sent via `POST /api/game/difficulty`.
 - Validation: The server rejects difficulty changes outside of SETUP state and rejects invalid difficulty values. On new game, the client sends the currently selected difficulty.

STATE REASONING

This feature requires real state reasoning because:

1. The AI's behavior is fundamentally different at each level — not just a parameter tweak but different algorithms (no queue vs queue vs scored queue with checkerboard).
2. Difficulty must be consistent within a game — changing mid-battle would be unfair and confusing. The server enforces this via state checks.
3. The scoreboard records difficulty per game, so players can compare their performance across difficulty levels.
4. Legacy game state files (without a difficulty field) are handled gracefully with a default of 'medium'.

ANNOTATED PROMPT LOG

Prompt 1:

"Implement three AI difficulty levels for Battleship: Easy (pure random), Medium (existing hunt/target), and Hard (smarter targeting). Each should be a separate function. Add a difficulty selector in the UI header."

Why I asked this:

Difficulty selection is a standard game feature that demonstrates real state reasoning. Each level requires a different AI algorithm, not just a parameter change.

What AI produced:

Three separate functions (``runComputerTurnEasy``, ``runComputerTurnMedium``, ``runComputerTurnHard``) and a dispatcher. UI buttons with active state styling.

Issue discovered:

The Easy AI initially still used ``addNeighborsToTargetQueue`` — it would queue neighbors but then ignore them on the next call since it always went to random. This caused the queue to grow indefinitely on Easy, wasting memory.

Prompt 2:

"Fix Easy AI: it should NOT call `addNeighborsToTargetQueue` at all. Easy mode should have zero targeting intelligence."

Why I asked this:

Even though Easy AI didn't USE the queue, adding to it was wasteful and semantically wrong. Pure random means no memory whatsoever.

What AI produced:

Removed the ``addNeighborsToTargetQueue`` call from ``runComputerTurnEasy()``. The function now only pushes to ``yourHits`` or ``yourMisses`` and transitions state — no queue interaction at all.

Prompt 3:

"For Hard AI, implement checkerboard hunting pattern and scored targeting that prioritizes cells continuing a line of hits."

Why I asked this:

Hard AI should feel noticeably smarter than Medium. Checkerboard hunting is a known optimal strategy (it guarantees finding the 2-length ship in at most 50 shots) and line-continuation scoring makes the AI "think" about ship orientation.

What AI produced:

Hard AI with checkerboard pattern for hunt mode (prefers cells where $(\text{row} + \text{col}) \% 2 == 0$), and a scoring system for target mode that gives +2 for each adjacent hit and +3 bonus for two consecutive hits in the same direction.

Prompt 4:

"Only allow difficulty changes during SETUP phase. Add a server endpoint for changing difficulty and disable the buttons during gameplay."

Why I asked this:

Allowing mid-game difficulty changes would undermine the game's integrity. A player could switch to Easy when losing.

What AI produced:

`POST /api/game/difficulty` endpoint with state validation, client-side `setDifficultyEnabled()` function that disables/enables buttons based on game state.

MANUAL CHANGES

1. Added a DIFFICULTY constant object (similar to STATE) for string validation to avoid magic strings throughout the code.
2. Added backward compatibility: when loading a legacy game-state.json without a difficulty field, the server defaults to 'medium' and re-saves.
3. Ensured the new-game endpoint accepts difficulty in the request body AND falls back to the current difficulty if not provided — so the UI can send the selected difficulty when starting a new game.
4. Verified that the Hard AI's checkerboard fallback (when all checkerboard cells are fired) correctly falls back to non-checkerboard cells instead of returning without firing.
5. Tested that difficulty is correctly recorded in scoreboard game history entries by tracing the data flow from game.difficulty through recordGameResult() to scoreboard.gameHistory entries.

=====

==

END OF DOCUMENT

=====

==